SOURCE-SPARK

Open-Source Discovery & Contribution Platform

Submitted By:

Priyanshu Anand - AP24110010609

Nirali - AP24110010636

Tushar Sharma - AP24110010643

Ratnesh Bhandari - AP24110010649

Raghav Mukherjee - AP24110010807

INTRODUCTION

Source-Spark is a versatile and user-friendly web application designed to bridge the gap between open-source repositories and potential contributors. Suitable for junior developers, experienced engineers, and open-source maintainers, it offers a curated database of high-signal repositories that can be filtered using a simple, intuitive interface based on tech stack, programming language, and difficulty level. Users can fully tailor their discovery experience, while robust backend features like JWT authentication and secure data handling protect user integrity. Source-Spark also integrates essential open-source roadmaps through an interactive Sprint Sheet and tracks mentorship programs (like GSoC and Outreachy) in a single workspace. Its flexibility, focus on reducing choice paralysis, and powerful curation features make it an ideal choice for developers seeking an efficient open-source onboarding solution.

SCENARIO Based Case Study

Meet Ratnesh, a computer science student with a demanding academic schedule who values efficiency in his technical growth. Ratnesh wants to build his resume and gain real-world experience, which requires him to stay updated on open-source ecosystems, find approachable repositories, and track his contributions.

- **The Problem:** Ratnesh experiences "choice paralysis" on GitHub due to the sheer volume of uncurated repositories.
- **The Solution:** He discovers Source-Spark.
- **User Registration:** Ratnesh registers and logs in securely to keep his bookmarks and sprint progress private.
- **Repository Discovery:** He accesses the OSS Projects catalog, filtering specifically for "React" and "Beginner-friendly" difficulty to find issues he can tackle immediately.
- **Sprint Tracking:** He uses the OSS Sprint Sheet to track his daily progress across 32 curated educational modules.
- **Mentorship:** Ratnesh uses the OSS Programs tab to find and track application windows for programs like the MLH Fellowship.

SYSTEM REQUIREMENTS

1. Software Requirements

These are the essential tools required to develop and run the Source-Spark application:

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Node.js (v16+) & npm (v8+):** Provides the runtime environment for server-side logic and package management.
- **React.js (via Vite):** A JavaScript library and build tool for building fast, responsive user interfaces.
- **Express.js & MongoDB:** Express provides the RESTful API framework, while MongoDB acts as the NoSQL database for flexible metadata storage.
- **Tools:** Google Chrome/Firefox, Visual Studio Code.

2. Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better.
- **RAM:** Minimum 8 GB (16 GB recommended) to handle concurrent local development servers.
- **Storage & Display:** At least 1 GB free space and a 1366x768 display for optimal UI rendering.

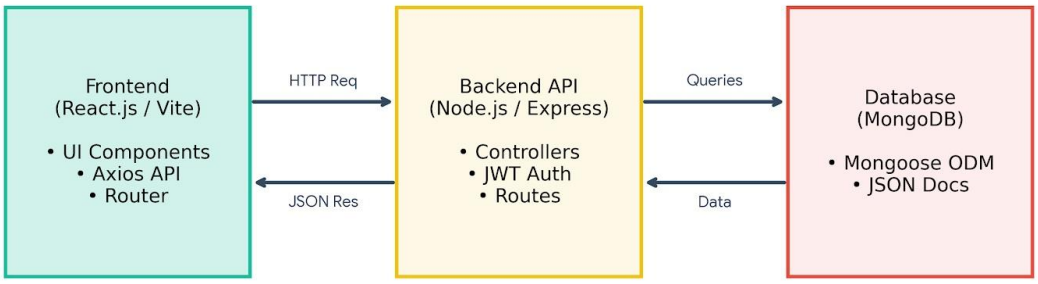
PROJECT ARCHITECTURE

The architecture is divided into decoupled client and server layers. The Frontend (Client) is built with React and Vite, housing components like the Dashboard, OSS Projects, and OSS Sprint Sheet. This communicates via HTTP requests (using Axios) to the Backend API (Node.js/Express). The backend handles business logic, JWT authentication, and routes, and it queries the MongoDB Database via Mongoose ODM to store and retrieve user profiles and repository metadata.

TECHNICAL ARCHITECTURE

The technical flow begins with a user interacting with the Source-Spark frontend to search for a repository or update a sprint task. The React frontend sends an asynchronous request with a JWT authorization header to the Express backend. The authController or projectController processes the request, sanitizes inputs, and communicates securely with the MongoDB database to fetch or update the document. Once the database operation is complete, the backend sends a JSON response back to the frontend, which dynamically updates the UI without a page reload.

Source-Spark: System Architecture

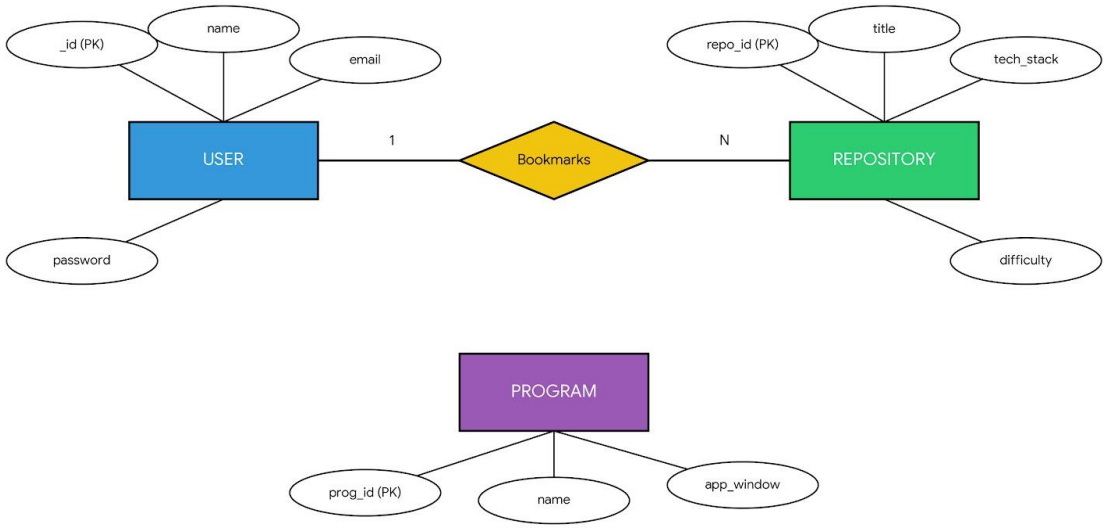


ER DIAGRAM

The Entity-Relationship diagram outlines the database schema for Source-Spark:

- **User Entity:** Attributes include _id (Primary Key), name, email, password (hashed), and an array of savedProjects.
- **Repository Entity:** Attributes include repo_id (Primary Key), title, description, tech_stack (Array of strings), difficulty, and github_url.
- **Program Entity:** Attributes include program_id, name, description, and application_window.
- **Relationships:** A 1-to-N relationship exists where one User can bookmark multiple Repositories.

Source-Spark: Entity-Relationship (ER) Diagram



Key Features

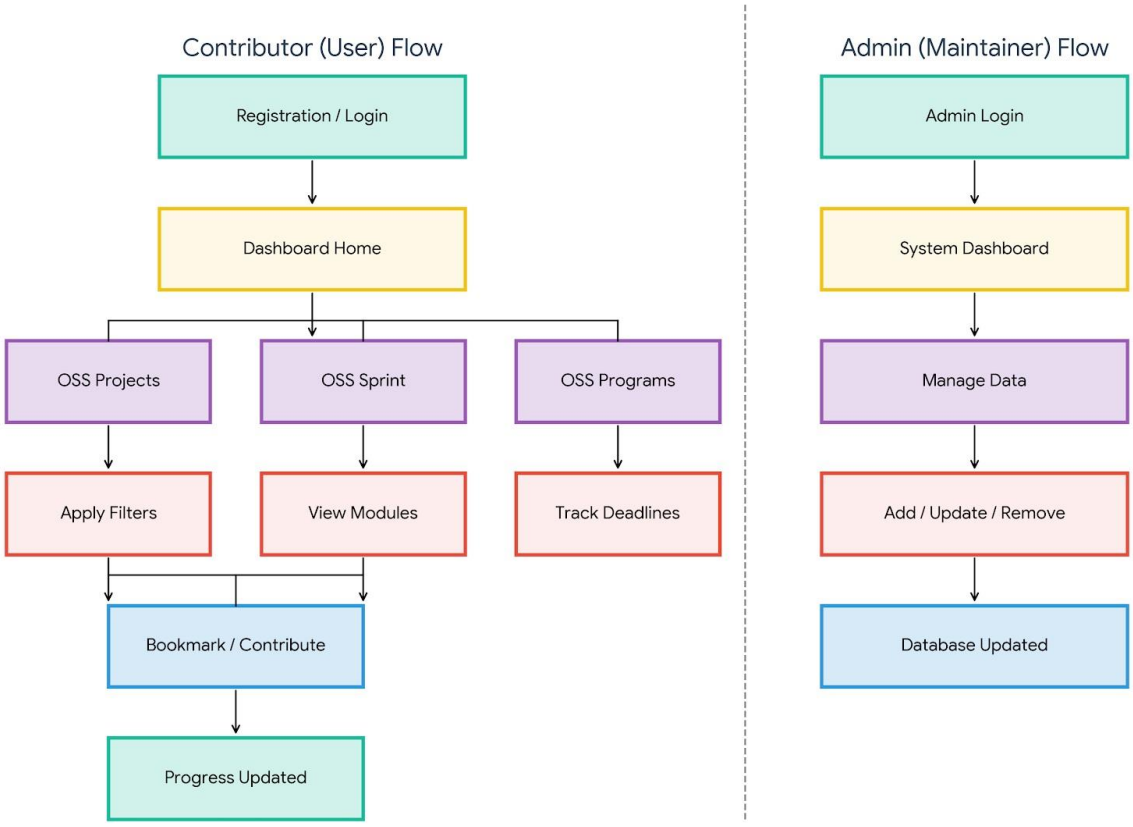
- **User Registration and Authentication:** Secure login system utilizing bcrypt for password hashing and JSON Web Tokens (JWT) for stateless sessions.
- **Precision Filters:** Allows users to scan thousands of open-source projects by narrowing them down by activity, competition, language, and stack.
- **OSS Sprint Sheet:** An interactive 32-module roadmap tracking system that helps contributors learn prerequisites and stay consistent.
- **OSS Programs Hub:** A centralized directory for major open-source fellowships (GSoC, LFX, Outreachy) with direct links and descriptions.

ROLES AND RESPONSIBILITIES

- **User (Contributor):** Responsible for creating an account, setting their tech stack preferences, filtering and finding repositories to contribute to, and updating their progress on the Sprint Sheet.
- **Admin (Maintainer):** Responsible for system management, curating high-signal repositories to add to the database, ensuring links to OSS programs are up-to-date, and monitoring system security.

User Flow

Source-Spark: User Flows



Contributor Flow

● Registration & Account – Create and manage account details securely to track open-source progress.

● Search & Discovery – Find repositories by filtering through tech stack, language, and difficulty levels.

● Sprint Tracking – Update and manage learning milestones using the 32-module OSS Sprint Sheet.

● Mentorship Programs – Explore opportunities and track application windows for programs like GSoC or Outreachy.

● Contribution & Bookmarking – Save high-signal repositories for later or navigate directly to GitHub to submit pull requests.

Admin Flow

● System Management – Oversee platform performance, database health, and backend security.

● User Management – Monitor registered contributors and maintain secure authentication workflows.

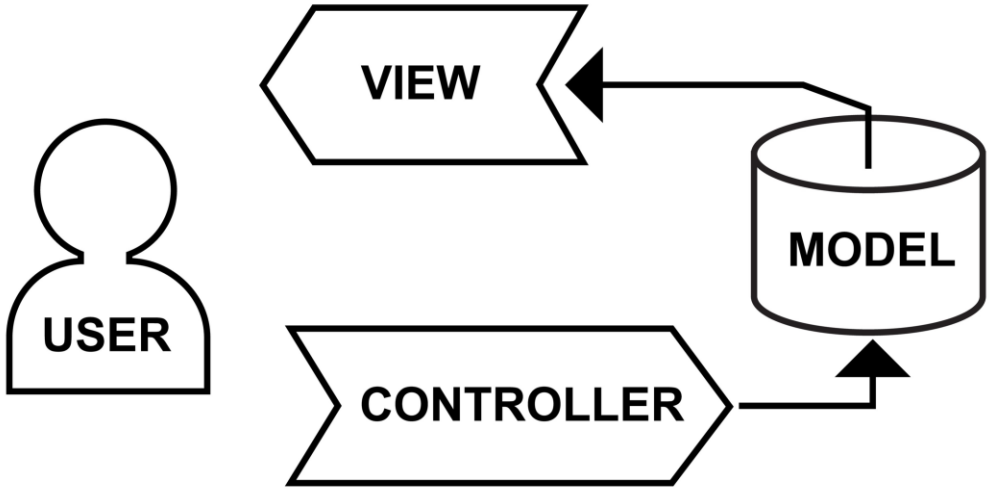
● Repository & Program Management – Curate and add high-signal open-source projects and ensure external program links and deadlines are accurate.

MVC PATTERN

The Source-Spark application follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates the application into three interconnected layers. This separation allows for high modularity, easier maintenance, and scalability as the open-source repository database grows.

Model Layer (Data Layer) The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas (such as Users, Repositories, and OSS Programs) and the operations performed on the database using those schemas. The models are implemented using Mongoose, which provides a rigid, schema-based solution to model the application's data for the MongoDB cluster.

View Layer (Presentation Layer) In the context of Source-Spark's decoupled full-stack architecture, the View is implemented by the React.js frontend. It is responsible for rendering the interactive user interface—such as the filtering dashboards, repository cards, and the OSS Sprint Sheet—by consuming and formatting the JSON data it receives from the backend.



MODEL - VIEW - CONTROLLER PATTERN

Controller Layer (Business Logic Layer) The Controller layer acts as an intermediary between the View (React frontend) and the Model (MongoDB). Implemented via Express.js route handlers (e.g., authController.js), it receives incoming HTTP requests, processes the input (which includes JWT validation, password hashing, and search filtering), calls the appropriate methods from the Mongoose models, and then returns a structured JSON response back to the client.

Advantages of Using MVC in This Project

● **Separation of Concerns:** Each layer has a clearly defined responsibility, isolating the database logic from the UI rendering, which improves readability and maintainability.

● **Scalability:** New features, such as adding AI tagging or semantic search, can be added easily by creating new routes, controllers, and models without breaking the existing frontend.

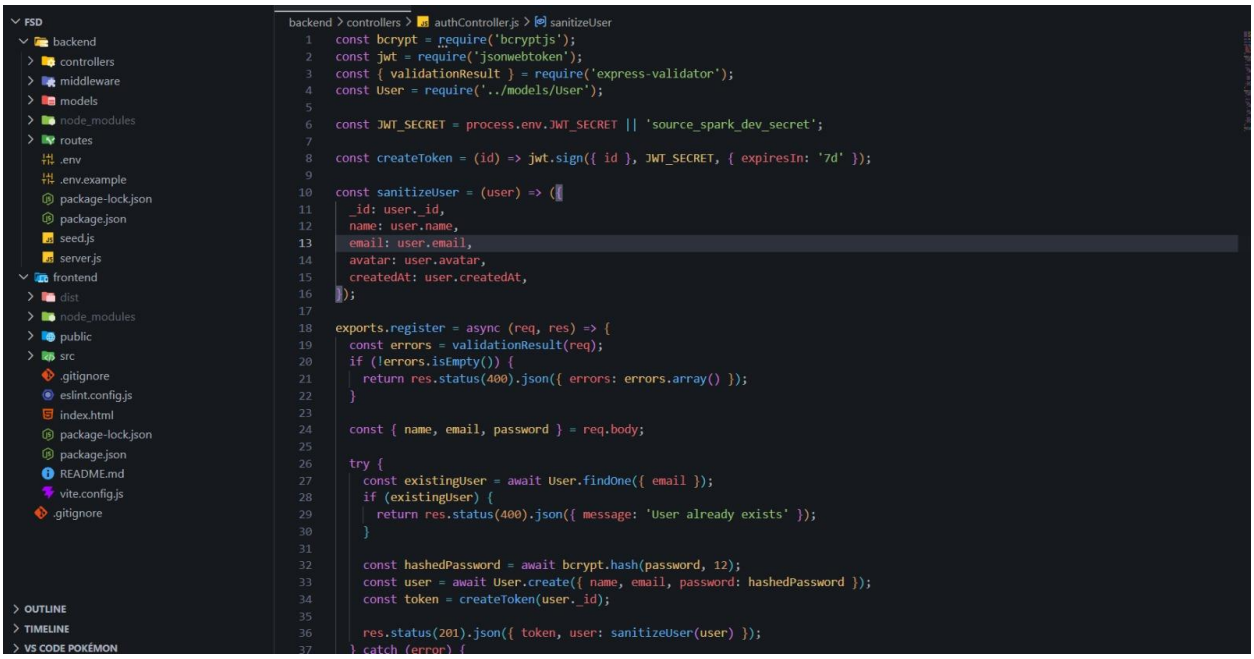
● **Reusability:** Business logic in the controllers and data validation in the models can be reused across multiple parts of the application or by different API endpoints.

● **Testing:** Each layer can be tested independently, allowing developers to write isolated unit tests for the Express controllers and Mongoose models.

● **Collaboration-Friendly:** Multiple developers can work simultaneously on different layers (e.g., one building the React UI while another optimizes the MongoDB controllers) without encountering merge conflicts.

Project Setup And Configuration

1. Create a main project folder named Source-Spark with two sub-folders: frontend and backend.
2. **Frontend Setup:** Navigate to the frontend folder, run npm create vite@latest . --template react, and install dependencies via npm install. Start the server with npm run dev.
3. **Backend Setup:** Navigate to the backend folder, initialize with npm init -y, and install necessary packages (npm i express mongoose jsonwebtoken bcryptjs dotenv). Create directories for models, controllers, and routes.



Backend Structure

The backend is built using Node.js and Express.js, and it follows a well-defined directory structure:

- **controllers/**: This directory contains the business logic for handling incoming requests. Instead of putting all the logic directly inside the route definitions, it's separated here.
    - authController.js: This file specifically handles authentication-related logic, such as user registration and login. It includes functions like exports.register which processes user data, hashes passwords (using bcrypt), and generates JWTs (JSON Web Tokens). It also often includes utility functions like sanitizeUser to ensure sensitive data (like the password hash) isn't sent back to the client.
- **middleware/**: This directory holds middleware functions. Middleware functions are executed in the middle of the request-response cycle. They can modify the request or response objects, end the request-response cycle, or call the next middleware function in the stack.
    - A common example here would be a function to verify the JWT attached to incoming requests to protect certain routes.
- **models/**: This directory defines the data schemas used by the application, specifically using Mongoose to interact with the MongoDB database.
    - User.js: This defines the structure of a user document in the database (e.g., _id, name, email, password, savedProjects).
- **routes/**: This directory defines the API endpoints and maps them to the appropriate controller functions.
    - For example, an authRoutes.js file would map the /api/auth/register path to the register function inside the authController.js.
- **config/** (Often present): Though not explicitly listed in the minimal structure provided earlier, this directory typically holds configuration files, such as the logic for connecting to the MongoDB database.
- **.env**: This file stores environment variables, which are sensitive configuration details that shouldn't be hardcoded into the source code (e.g., database connection strings, JWT secret keys, port numbers).
- **server.js** (or index.js): This is the main entry point for the backend application. It initializes the Express app, connects to the database, sets up global middleware (like CORS or JSON parsing), and mounts the route handlers defined in the routes/ directory.

- **package.json**: This file manages the project's dependencies (the packages installed via npm, like express, mongoose, bcryptjs, etc.) and defines scripts for running or building the application.

Frontend Architecture & Structure

The frontend is built primarily using **React.js** and initialized via **Vite**. Vite is a modern build tool that offers significantly faster development server start times and hotter module replacement (HMR) compared to older tools like Create React App.

The core structure within the frontend/ directory (typically inside a src/ folder) looks like this:

- **components/**: This directory is the heart of React's modular design. It contains reusable UI elements (Atomic Components).
    - *Examples:* You might have a ProjectCard.jsx (which displays the details of a single repository), a FilterSidebar.jsx (for the tech stack/language filters), a Button.jsx, or a Badge.jsx (to show difficulty levels). By breaking the UI down into these smaller pieces, the code remains clean, testable, and easy to reuse across different pages.
- **pages/** (or views/): This directory contains the main layout components that represent entire "screens" or routes within the application. These pages usually assemble various components from the components/ directory.
    - *Examples:*
        - Dashboard.jsx: The main landing area after a user logs in, showing summaries and progress.
        - OSSProjects.jsx: The page containing the complex filtering interface and the grid of repository cards.
        - OSSSheet.jsx: The view displaying the 32-module interactive roadmap/sprint tracker.
        - OSSPrograms.jsx: The page listing mentorship programs like GSoC and Outreachy.
        - Login.jsx & Signup.jsx: The authentication screens.
- **context/** (Global State Management): For state that needs to be accessed by many different components (like whether a user is currently logged in, or their profile data), React's Context API is used.
    - *Example:* An AuthContext.jsx file would wrap the entire application. It stores the JWT token and user details. This prevents "prop drilling"

(passing data down through many layers of components that don't actually need the data themselves).

- **App.jsx**: This is the root component that acts as the container for the entire application. It typically holds the routing logic.
- **main.jsx** (or index.js): This is the entry point where the React application is rendered into the actual HTML DOM (usually into a div with the id "root").

Key Frontend Concepts in Source-Spark

1. **Routing (react-router-dom)**:
    a. The application uses React Router to manage navigation. When a user clicks "OSS Projects" in the sidebar, the URL changes, and React Router swaps out the Dashboard component for the OSSProjects component *without* reloading the page.
    b. **Protected Routes:** React Router is also used to enforce security. A custom wrapper (e.g., <PrivateRoute>) checks the AuthContext to see if a valid JWT exists. If not, it redirects the user back to the /login page, preventing unauthorized access to the dashboard.
2. **API Integration (axios)**:
    a. To communicate with the Node.js/Express backend, the frontend uses Axios, a promise-based HTTP client.
    b. **Interceptors:** A common practice is to configure an Axios "interceptor." This automatically intercepts every outgoing request from the frontend and attaches the user's JWT token to the Authorization header. This ensures the backend knows *who* is making the request without having to manually add the token to every single API call in the code.
3. **State Management & Lifecycle (useState, useEffect)**:
    a. **useState**: Used to manage local component state, such as what filters are currently selected on the OSS Projects page or what the user has typed into the search bar.
    b. **useEffect**: Used to handle "side effects," most notably fetching data. For instance, when the OSSProjects component first loads (mounts), a useEffect hook triggers an Axios request to /api/projects to fetch the list of repositories from the backend and then updates the local state to render them on the screen.
4. **Responsive Design**:
    a. The frontend is built to be responsive, meaning it adapts its layout based on the screen size (mobile, tablet, desktop). This is typically achieved using CSS Grid and Flexbox, ensuring the dashboard looks clean whether viewed on a phone or a large monitor.

DATABASE DEVELOPMENT

The Source-Spark application uses MongoDB, a NoSQL database, to store user and repository data. Mongoose is used as an Object Data Modeling (ODM) library to enforce data structure and provide a straightforward way to interact with the database.

1. Setup and Connection

Install Mongoose: First, you need to install the Mongoose package in your backend directory:

Bash
```
npm install mongoose
```

Database Connection (db/config.js or similar): The connection to the MongoDB database (whether local or hosted on MongoDB Atlas) is established using mongoose.connect. This connection string should be stored securely as an environment variable (MONGO_URI).

JavaScript
```
const mongoose = require('mongoose');
const dotenv = require('dotenv');
dotenv.config();

const connectDB = async () => {
    try {
        await mongoose.connect(process.env.MONGO_URI, {
            useNewUrlParser: true,
            useUnifiedTopology: true,
        });
        console.log('MongoDB Connected...');
    } catch (error) {
        console.error('Database connection failed:', error);
        process.exit(1); // Exit process with failure
    }
};
```

```
module.exports = connectDB;
```

Note: Options like useNewUrlParser and useUnifiedTopology are often standard in older Mongoose setups, though newer versions may handle these automatically.

2. Creating Schemas and Models

Mongoose schemas dictate the structure of the documents stored in a specific MongoDB collection. They enforce data types, validation rules (like required or unique), and default values.

Based on the ER diagram and features discussed, here are the detailed schema definitions for Source-Spark.

A. User Schema (models/User.js)

This schema handles user authentication and profile data. It includes a reference array to link a user to their bookmarked repositories.

JavaScript
```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
    name: {
        type: String,
        required: [true, 'Please provide a name'],
        trim: true
    },
    email: {
        type: String,
        required: [true, 'Please provide an email'],
        unique: true,
        lowercase: true,
        match: [
            /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
            'Please fill a valid email address'
        ]
    },
    password: {
        type: String,
```

```
        required: [true, 'Please provide a password'],
        minlength: 6,
        select: false // Ensure password isn't returned in generic queries
    },
    avatar: {
        type: String,
        default: 'default-avatar.png'
    },
    savedProjects: [{
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Project' // Reference to the Project model
    }],
    createdAt: {
        type: Date,
        default: Date.now
    }
});

module.exports = mongoose.model('User', userSchema);
```

B. Repository/Project Schema (models/Project.js)

This schema defines the structure for the open-source repositories indexed by the platform.

JavaScript

```
const mongoose = require('mongoose');

const projectSchema = new mongoose.Schema({
    title: {
        type: String,
        required: [true, 'Project title is required'],
        trim: true
    },
    description: {
        type: String,
        required: [true, 'Project description is required']
    },
    tech_stack: [{
        type: String // e.g., 'React', 'Node.js', 'Python'
```

```
    }],
    difficulty: {
        type: String,
        enum: ['Beginner', 'Intermediate', 'Advanced', 'All Levels'],
        default: 'All Levels'
    },
    github_url: {
        type: String,
        required: [true, 'GitHub URL is required'],
        match: [
            /^(https?:\/\/)?(www\.)?github\.com\/[a-zA-Z0-9-]+\/[a-zA-Z0-9_.-]+$/,
            'Please provide a valid GitHub repository URL'
        ]
    },
    active_contributors: {
        type: Number,
        default: 0
    },
    createdAt: {
        type: Date,
        default: Date.now
    }
});

// Create indexes for faster searching/filtering
projectSchema.index({ tech_stack: 1 });
projectSchema.index({ difficulty: 1 });

module.exports = mongoose.model('Project', projectSchema);
```

C. OSS Program Schema (models/Program.js)

This schema stores information about various open-source mentorship programs (like GSoC).

JavaScript

```
const mongoose = require('mongoose');

const programSchema = new mongoose.Schema({
    name: {
```

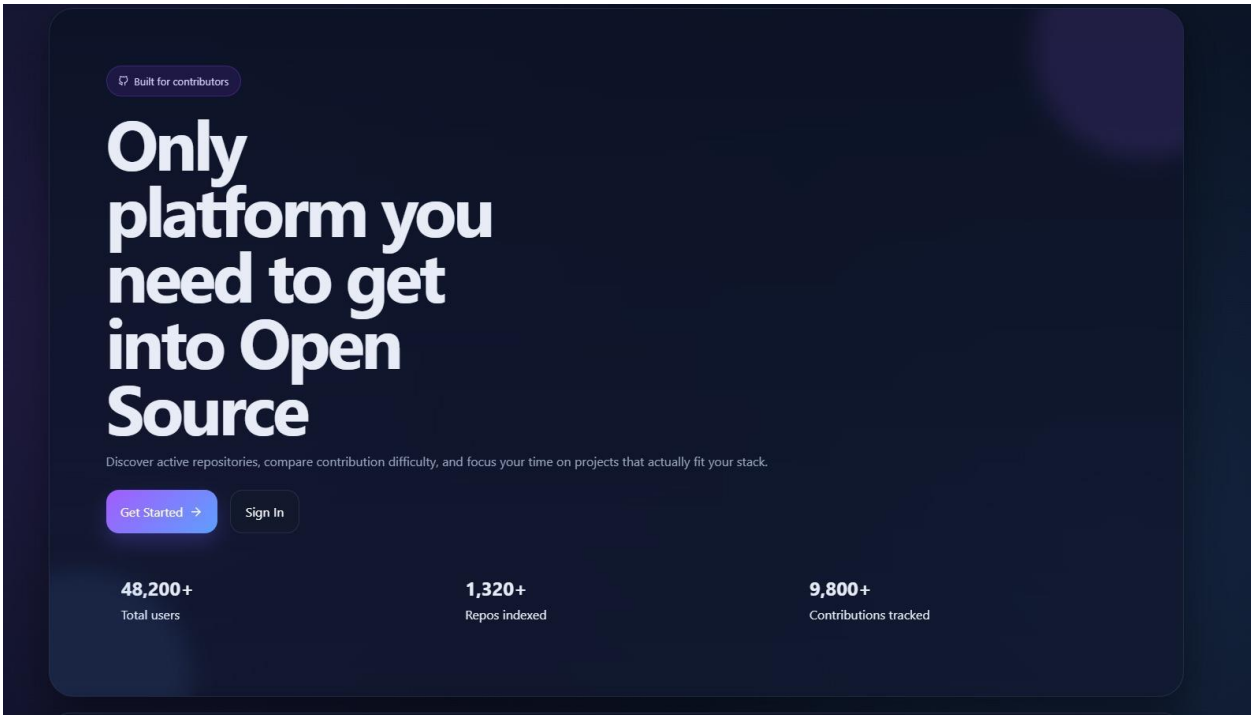
```
        type: String,
        required: true,
        trim: true
    },
    description: {
        type: String,
        required: true
    },
    application_window: {
        type: String, // e.g., "Jan - Feb", or specific dates
        required: true
    },
    link: {
        type: String,
        required: true
    }
});

module.exports = mongoose.model('Program', programSchema);
```

3. Best Practices Implemented

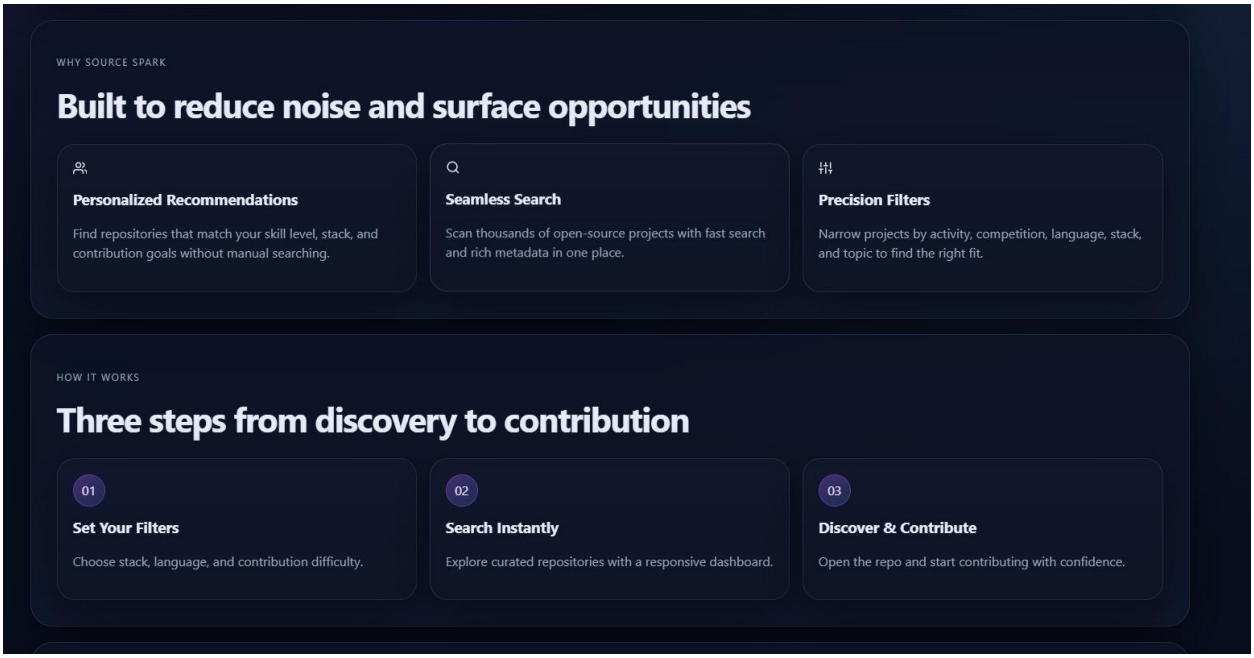
- **Validation:** Schemas use built-in Mongoose validators (like required, unique, enum, and Regex match for emails and URLs) to ensure data integrity before it hits the database.
- **References (ref):** The savedProjects array in the User schema uses mongoose.Schema.Types.ObjectId to create relationships between collections, mimicking relational behavior in a NoSQL environment.
- **Indexing:** The projectSchema.index() method is used to create indexes on frequently queried fields like tech_stack and difficulty. This drastically improves the performance of the search and filtering features on the frontend.
- **Security:** The select: false option on the user's password field ensures that when a user document is queried, the password hash isn't accidentally sent to the client unless explicitly requested (e.g., during login validation).

Output Screenshots

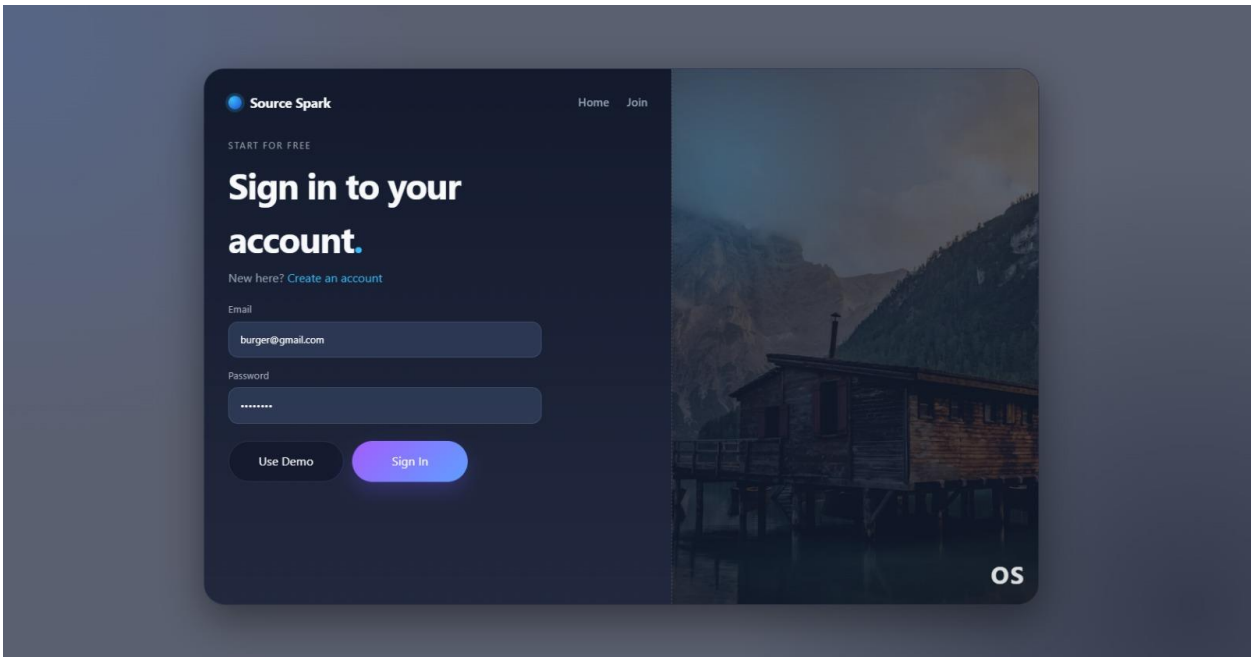
Landing Page



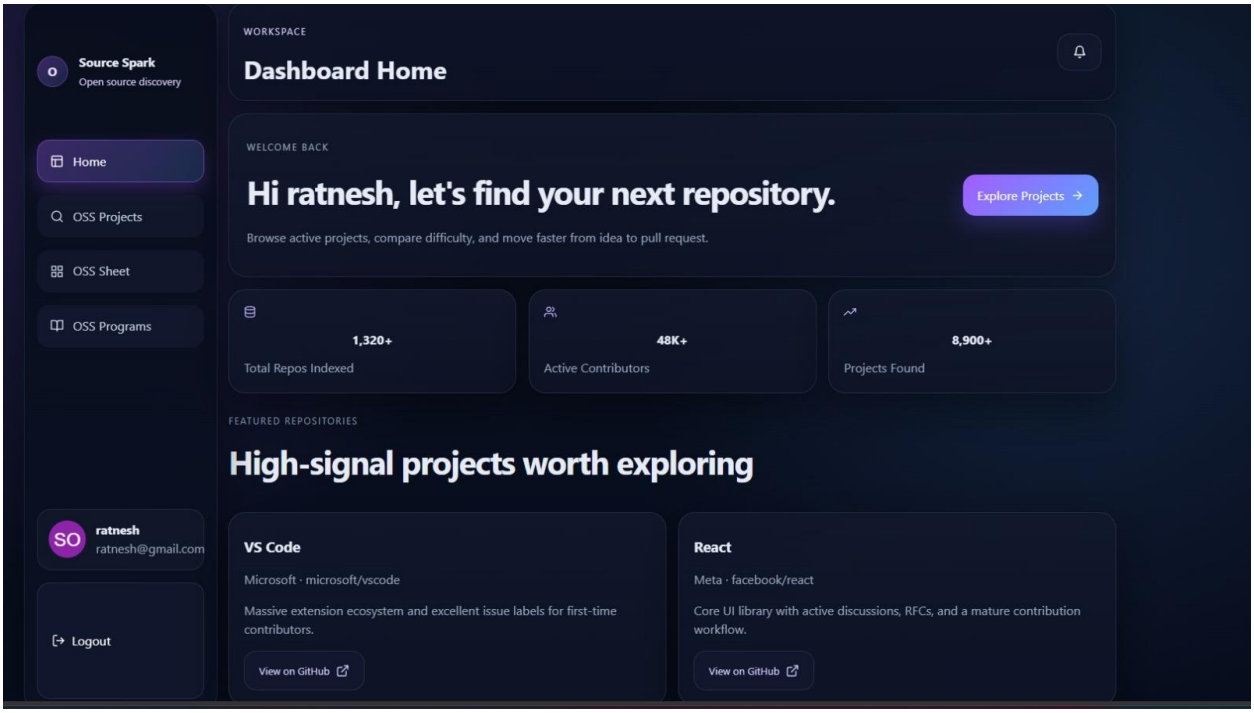
Features Section



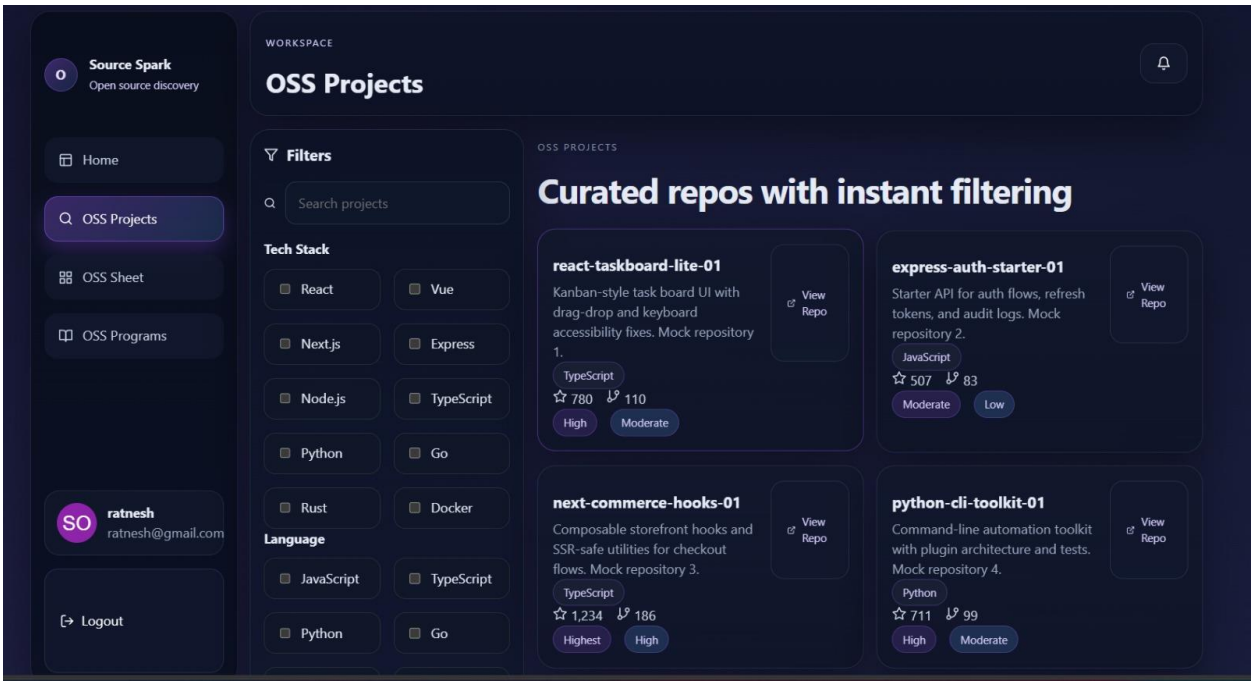
User Login Page



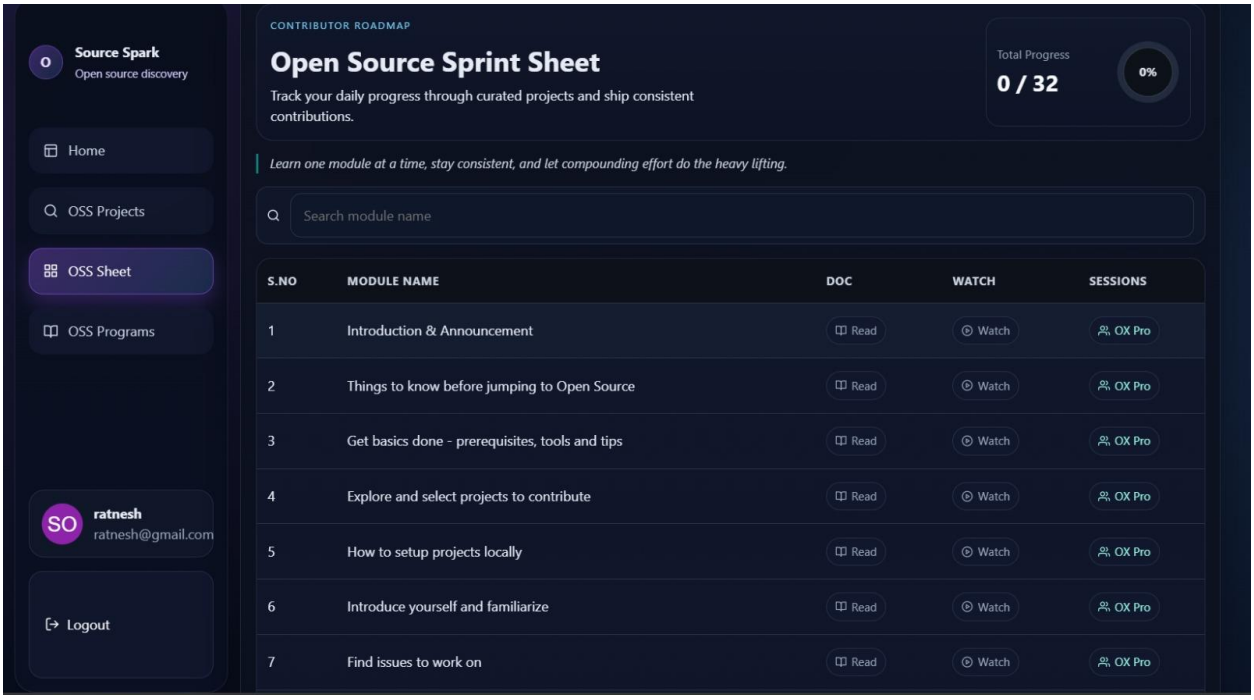
Dashboard Home



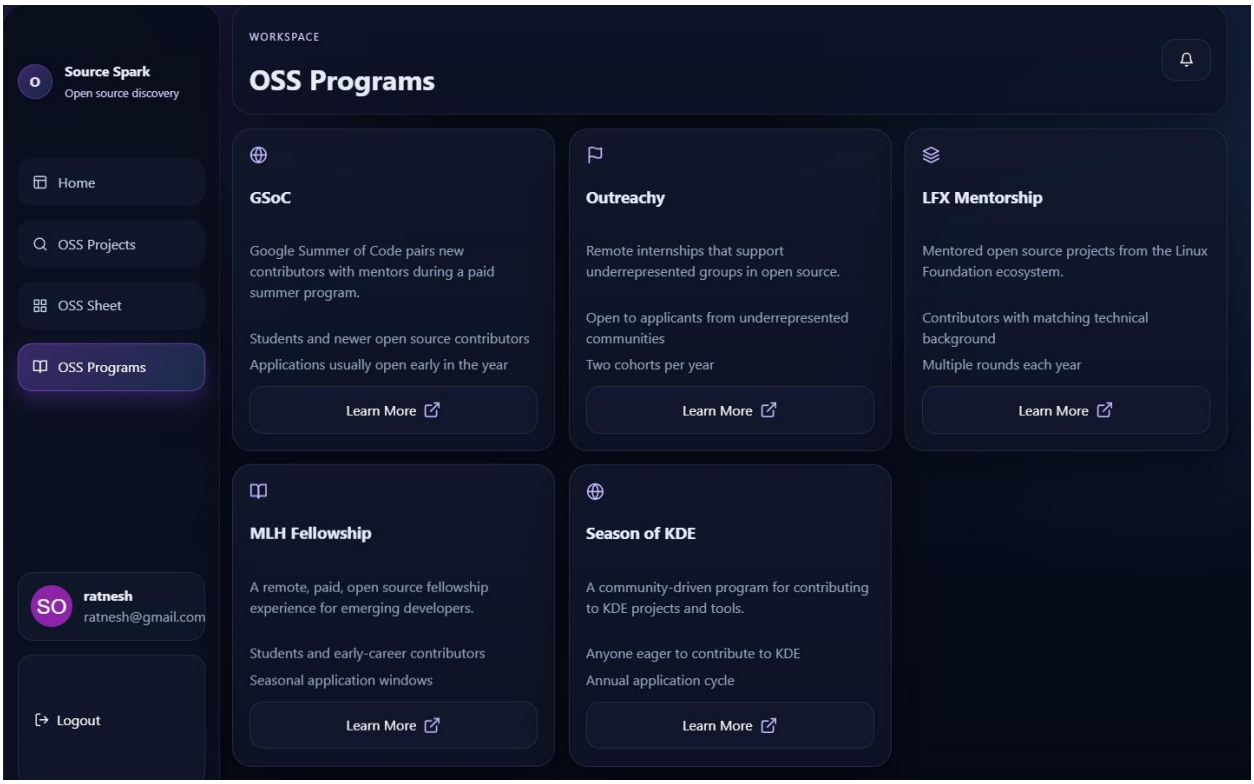
OSS Projects / Repository Discovery Page



OSS Sprint Sheet



OSS Programs Directory



Project Overview:

https://1drv.ms/p/c/4d586745065d093c/IQBjvUIQ80uOQqwK0WMSmiA1Ad1GjpIgTCJEcXOm2O97qt8?e=vNJrXP

Project Code Explanation:

https://1drv.ms/p/c/4d586745065d093c/IQCAo34Da0KTQ45I3rE4DvWaAbjMXZbv02jCOez8Dchf_dw?e=3bd7mG